CECILLE'S N'STYLE BOUTIQUE MANAGEMENT SYSTEM

DIAGRAM REVISION TASK GUIDE

Use Case · Design Class · Domain Class Diagrams

For internal review and correction by the group

Ilanga · Palanca · Sintos

Ateneo de Davao University

System Integration and Architecture 2 | S.Y. 2025–2026, 2nd Semester

Based on full manuscript review + codebase cross-reference

April 30, 2026

✓ **GREEN** = Correct — no change needed (or minor elaboration only)

✗ **RED** = Wrong — must be corrected before submission

→ **BLUE** = Action to take when redrawing the diagram

PART 1 — USE CASE DIAGRAMS (2.3.1 to 2.3.8)

Each subsection below covers one use case diagram. If a diagram is correct, it is marked as such and elaborated. If it needs changes, the exact corrections are listed.

2.3.1 — User and Role Management System (RBAC)

✓ Role management use cases are correct: Create Custom Role, View Role, Delete Role, Modify Role Access Rights — all match the system.

✓ Create Employee/User Profile (Step 1) with <<include>> sub-forms (Account Details, Employment Details, Personal Information, Pay Rate, Government Details, Emergency Contact) — structure is acceptable.

✓ Manage Employee Documents (Step 2) with <<include>> for Upload Documents — correct.

✗ Login and Logout use cases are MISSING from this diagram entirely.

✗ There is only one actor (Store Owner / Admin). A second actor is needed for Login and Logout.

WHAT TO ADD:

→ Add a second actor labeled 'User' (stick figure, left side of diagram).

→ Draw a line from [User] → Login

→ Draw a line from [User] → Logout

→ These two use cases do not belong only to the Admin — any system user logs in and out.

CORRECTED ACTOR AND USE CASE ADDITIONS:

```
Actor: [User]
  [User] ──▶ Login
  [User] ──▶ Logout

Actor: [Store Owner / Admin]   (keep all existing connections)
```

2.3.2 — Inventory Management System

✓ Product creation, stock-in (manual and from bale), stock-out, damaged item recording, item repair, shrinkage recording, barcode/QR generation, and inventory reports — all correct in structure.

✓ Actors and connections for Store Manager and Clerk are appropriate.

✗ Low Stock Alert is shown as the SYSTEM automatically notifying the Manager. This is factually wrong.

The system has a low-stock query endpoint (GET /inventory/alerts/low-stock). It returns a list of products below threshold only when a user actively checks it. No automatic notification is sent to anyone.

WHAT TO CHANGE:

→ Remove any arrow showing the system pushing a Low Stock Alert automatically.

→ Replace with a use case: 'View Low Stock Alerts'

→ Connect this use case to the [Store Manager / Owner] actor with a normal association line.

→ This is a user-initiated query, not a system-triggered event.

CORRECTED LOW STOCK ALERT:

```
BEFORE: [System] ──► Low Stock Alert ──► 'Notifies Manager'
AFTER:  [Store Manager / Owner] ──► View Low Stock Alerts
         (user opens the alerts page; system returns the list)
```

2.3.3 — Sales Management System (POS)

✓ Barcode/QR scanning, compute totals, accept payment, print receipt, process sales return — all correct.

✓ Actors (Cashier, Admin) and their connections are appropriate.

✗ 'Add product to sales cart' — the system does not use a cart object. Items are added to a draft sale record stored in the database.

✗ The checkout process is shown as a single step. In the system, checkout is a separate action that converts a DRAFT sale to COMPLETED. Stock is only deducted at checkout, not when items are added.

WHAT TO CHANGE:

→ Rename 'Add product to sales cart' → 'Add Product to Draft Sale'

→ Split the sale process into two distinct use cases:

→ (1) 'Create Draft Sale' — system auto-creates this when POS is opened

→ (2) 'Checkout / Confirm Payment' — this finalizes, deducts stock, assigns receipt number

→ Add 'Cancel Draft Sale' connected to [Cashier] — this deletes the draft record from the database entirely, not just clears a screen.

CORRECTED STRUCTURE:

```
[Cashier] ──▶ Create Draft Sale
[Cashier] ──▶ Add Product to Draft Sale
                 <<include>>──▶ Scan Barcode / Search Product
[Cashier] ──▶ Checkout (Confirm Payment)
[Cashier] ──▶ Cancel Draft Sale
[Cashier] ──▶ Process Sales Return
```

2.3.4 — Purchasing Management System

✓ Create Bale Purchase Order, Receive Bale, View Purchase History — correct.

✓ Connecting the Manager actor to bale operations is appropriate.

✗ 'Add multiple order lines in one purchase order' — this DOES NOT EXIST in the system. One bale purchase record equals one category. There is no order line item table.

✗ Bale breakdown description implies the system passively converts items. In reality, the system AUTO-CREATES product records for Premium and Standard grades immediately when breakdown is saved.

✗ 'P.O. closes automatically after breakdown' — the po_status must be manually updated to COMPLETED. It does not close automatically.

WHAT TO CHANGE:

→ REMOVE the 'Add multiple order lines' use case completely.

→ In the breakdown use case, add a note or <<include>> showing: 'System auto-generates product records for Premium and Standard grades'

→ Low-grade and damaged items are recorded as counts only — no products are auto-created for them. Add this as a note.

→ Remove any notation showing the P.O. closing automatically.

CORRECTED BREAKDOWN USE CASE:

```
[Manager] ──▶ Process Bale Breakdown
                <<include>>──▶ Record Grade Quantities
                <<extend>>──▶ Record Damaged / Unsellable
                System auto-creates:
                  - 1 product for Premium grade
                  - 1 product for Standard grade
                  (low-grade and damaged: counted only, no product
created)
```

2.3.5 — Payroll Management System

✓ Create Payroll Period and Record Attendance are correct as starting points.

✓ Sync Attendance and Edit Inputs as related use cases are correct.

✗ 'Process Payroll' is shown as a single use case. Payroll has a 6-stage lifecycle — each stage is a separate, permission-controlled operation.

✗ 'Finalize Payroll → Owner Approves' — there is NO owner-specific approval gate. Any user with the correct permission (payroll.finalize) can finalize. Showing 'Owner' as the only approver is factually wrong.

✗ 'Generate Payslip PDFs' — the server does not generate PDFs. Payslip data is stored in the database. PDF rendering happens in the browser.

WHAT TO CHANGE:

→ Split 'Process Payroll' into 5 separate use cases:

→ 1. Compute Payroll

→ 2. Finalize Payroll (locks results, no more edits)

→ 3. Release Payroll (marks as disbursed)

→ 4. Void Payroll (cancels the run — can happen before release)

→ Label the actor for finalize/release as 'Authorized User' not 'Owner specifically.'

→ Remove 'Generate payslip PDFs' — replace with 'Payslip data stored; viewable by employee'

CORRECTED LIFECYCLE USE CASES:

```
[HR / Manager] ⟶ Create Payroll Period
[HR / Manager] ⟶ Sync Attendance to Inputs
[HR / Manager] ⟶ Edit Payroll Inputs
[Authorized User] ⟶ Compute Payroll
[Authorized User] ⟶ Finalize Payroll
[Authorized User] ⟶ Release Payroll
[Authorized User] ⟶ Void Payroll
```

2.3.6 — Supplier Management System

✓ Manage Supplier Profiles — correct.

✓ View Supplier List/Directory — correct.

✓ Select Supplier for Transaction — correct.

✓ All three use cases match the system exactly. Actor (Manager) and connections are appropriate.

No changes needed for this diagram.

2.3.7 — Category and Type Management System

✓ Create Product Category, Update Product Category, View Categories — correct.

✓ Create/Update Product Type under a Category — correct.

✓ Load Type Options based on selected Category — correct.

✗ The relationship between 'Link Product Type to Product Records' and the PRODUCT entity is shown with a solid line, implying a database foreign key. There is no FK in the database.

The product stores the type name as plain text in its subcategory field. The API validates this against the active types for that category at runtime — there is no database-level enforcement.

WHAT TO CHANGE:

→ Change the association line from Product to Category Type to a DASHED line.

→ Add label on the dashed line: 'validated at application layer (no FK)'

2.3.8 — Customer Management System

✓ Manage Customer Records — correct.

✓ View Customer Profile — correct.

✓ Search Customer Record — correct.

✓ Link Customer to Sale Transaction — correct.

✓ Track Purchase History — correct.

✓ Actor (Manager / Cashier) and connections are appropriate.

No changes needed for this diagram.

PART 2 — DESIGN CLASS DIAGRAMS (2.4.1 to 2.4.8)

Each design class diagram is reviewed below. The corrected class structures are provided in text notation that can be directly used to redraw each diagram.

2.4.1 — User and Role Management System (RBAC)

✓ ROLE class methods are correct: createRole(), updateRole(), assignPermissions(), deleteRole().

✓ USER class methods are correct: login(), logout(), createProfile(), updateProfile().

✓ The existence of a relationship between ROLE and USER is conceptually correct.

✗ ROLE class has '+String permissions' as a direct attribute. Permissions are NOT a column on the roles table. They are a separate entity linked through a junction table.

✗ USER class has '+int role_id' as a direct attribute. There is no role_id column on the users table. Users link to roles through a user_roles junction table (many-to-many).

✗ Relationship is shown as 1 ROLE → 0..* USER (one-to-many). This is wrong. One user can have multiple roles; one role can be assigned to multiple users. It must be many-to-many.

<u>CORRECTED CLASS DIAGRAM (redraw as follows):</u>

```
┌──────────────────────────┐     *         *
┌──────────────────────────┐
│            ROLE          │──────────────────│        PERMISSION
│
├──────────────────────────┤  via role_perms
├──────────────────────────┤
│ +int role_id             │                  │ +int permission_id
│
│ +String role_name        │                  │ +String name
│
│ +String description      │                  │   (format:
module.action)   │
├──────────────────────────┤                  │ +String description
│
```

```
│ +createRole()                │
└──────────────────────────────┘

│ +updateRole()                │
│ +assignPermissions()         │
│ +deleteRole()                │
└──────────────────────────────┘

           *
           │  (via user_roles junction, many-to-many)
           *

┌──────────────────────────────┐
│            USER              │
├──────────────────────────────┤
│ +int user_id                 │
│ +String username             │
│ +String password_hash        │
│ +String full_name            │
│ +String email                │
│ +int is_active               │
├──────────────────────────────┤
│ +login()                     │
│ +logout()                    │
│ +createProfile()             │
│ +updateProfile()             │
└──────────────────────────────┘
```

→ Remove '+String permissions' from ROLE attributes.

→ Remove '+int role_id' from USER attributes.

→ Add PERMISSION as a new class.

→ Change ROLE-USER relationship from 1:0..* to *:* (many-to-many).

→ Label the ROLE-USER association: 'via user_roles'

→ Label the ROLE-PERMISSION association: 'via role_permissions'

2.4.2 — Inventory Management System

✓ PRODUCT class exists — correct.

✓ Stock transaction record entity — correct in concept.

✓ Damaged item entity — correct in concept.

✓ Stock adjustment entity — correct in concept.

✗ PRODUCT class is missing critical bale-linked fields.

✗ STOCK_IN_RECORD should be renamed INVENTORY_TRANSACTION — it covers all movement types (IN, OUT, ADJUST, RETURN), not just stock-in.

✗ DAMAGED_ITEM should be renamed DAMAGED_INVENTORY to match the actual table name.

ADD these fields to the PRODUCT class:

→ +String item_code (boutique-specific identifier)

→ +int bale_purchase_id (FK — which bale this product came from, null if manual)

→ +String condition_grade (premium / standard / low_grade / damaged / unsellable)

→ +Decimal allocated_cost (bale cost ÷ total saleable items)

→ +String product_source (bale_breakdown or null for manually created products)

→ +String status (available / sold / damaged / reserved / archived)

RENAME and ADD to transaction class:

→ Rename STOCK_IN_RECORD → INVENTORY_TRANSACTION

→ Add: +String transaction_type (IN / OUT / ADJUST / RETURN)

→ Add: +int balance_after (stock count after the transaction)

→ Rename DAMAGED_ITEM → DAMAGED_INVENTORY

→ Add: +String damage_source (bale_breakdown / manual_damage / sales_return)

2.4.3 — Sales Management System (POS)

✓ SALES_TRANSACTION (Sale) entity with header fields — correct in concept.

✓ SALES_ITEM (Sale Item) entity linked to SALE — correct in concept.

✓ CUSTOMER linked to SALE — correct.

✓ SALES_RETURN entity — correct in concept.

✗ SALES_TRANSACTION is missing the DRAFT status — this is critical. POS creates a DRAFT sale first before any final record exists.

✗ SALES_ITEM is missing snapshot fields — the system stores a frozen copy of the product name, SKU, barcode, size, and color at the time of checkout so the record never changes even if the product is edited later.

✗ If a clearCart() method exists on the sale class — rename it to cancelDraft(). Clearing a cart is a UI concept; the system deletes the entire DRAFT record from the database.

ADD to SALE class:

→ +String status (DRAFT / COMPLETED / REFUNDED / CANCELLED)

→ +String receipt_no (assigned only at checkout, null while DRAFT)

ADD to SALE_ITEM class (snapshot fields):

→ +String product_name_snapshot

→ +String sku_snapshot

→ +String barcode_snapshot

→ +String size_snapshot

→ +String color_snapshot

→ These fields are stored at checkout and never change, even if the product is later edited.

RENAME method if present:

→ clearCart() → cancelDraft()

2.4.4 — Purchasing Management System

✗ BALE_PURCHASE_ORDER and BALE_PURCHASE_RECORD are shown as two separate classes. They are the SAME database table (bale_purchases). This must be merged into one class.

✗ ORDER_LINE_ITEM class must be REMOVED entirely. This table does not exist in the system. There are no line items on a bale purchase — one record equals one category.

✗ BALE_BREAKDOWN is missing key fields.

REMOVE:

→ Delete BALE_PURCHASE_ORDER class

→ Delete BALE_PURCHASE_RECORD class

→ Delete ORDER_LINE_ITEM class

REPLACE WITH one unified BALE_PURCHASE class:

```
┌──────────────────────────────────┐
│          BALE_PURCHASE           │
├──────────────────────────────────┤
│ +int id                          │
│ +String bale_batch_no            │  ← format: BALE-PO-0001
│ +int supplier_id                 │
│ +String supplier_name            │
│ +Date purchase_date              │
│ +String bale_category            │
│ +Decimal bale_cost               │
│ +int quantity_ordered            │
│ +int quantity_received           │
│ +Date expected_delivery_date     │
│ +Date actual_delivery_date       │
│ +String payment_status           │  ← PAID / PARTIAL / UNPAID
│ +String po_status                │  ← PENDING / ORDERED / RECEIVED
│                                  │     / COMPLETED / CANCELLED
└──────────────────────────────────┘
```

CORRECT BALE_BREAKDOWN class — add missing fields:

→ +int low_grade_items (was missing)

→ +int saleable_items (auto-computed: total - damaged)

→ +Decimal cost_per_saleable_item (auto-computed: bale_cost ÷ saleable_items)

→ +int encoded_by (FK to USER — who entered the breakdown)

ADD relationship between BALE_BREAKDOWN and PRODUCT:

→ BALE_BREAKDOWN ──[1]──generates──[0..2]── PRODUCT

→ One breakdown auto-creates up to 2 products: one for Premium grade and one for Standard grade.

→ Low-grade and damaged items are counted in the breakdown but do NOT generate product records.

2.4.5 — Payroll Management System

✓ PAYROLL_PROFILE entity — correct in concept.

✓ PAYROLL_PERIOD entity — correct in concept.

✗ PAYROLL_RECORD is a single class that combines two separate database tables. It must be split into PAYROLL_INPUTS (what goes in before computation) and PAYROLL_RUN_ITEM (the computed result per employee).

✗ PAYROLL_ADJUSTMENT does not exist as a separate table. Bonuses, deductions, and manual adjustments are fields on PAYROLL_INPUTS.

✗ PAYSLIP does not exist as a separate table. Payslip data is stored in PAYROLL_RUN_ITEM and PAYROLL_ITEM_LINE. The payslip is a view generated from those records — it is not a stored entity.

REMOVE:

→ Delete PAYROLL_ADJUSTMENT class

→ Delete PAYSLIP class

SPLIT PAYROLL_RECORD into two classes:

```
┌──────────────────────────────┐
│        PAYROLL_INPUTS        │  ← data entered before computation
├──────────────────────────────┤
│ +int payroll_period_id       │
│ +int user_id                 │
│ +int days_worked             │
│ +int overtime_hours          │
│ +int late_minutes            │
│ +int absent_days             │
│ +int paid_leave_days         │
│ +Decimal manual_bonus        │
│ +Decimal loan_deduction      │
│ +Decimal manual_deduction    │
└──────────────────────────────┘


┌──────────────────────────────┐
│       PAYROLL_RUN_ITEM       │  ← computed result per employee
├──────────────────────────────┤
```

```
| +int payroll_run_id          |
| +int user_id                 |
| +Decimal gross_pay           |
| +Decimal employee_sss        |
| +Decimal employee_philhealth |
| +Decimal employee_pagibig    |
| +Decimal withholding_tax     |
| +Decimal total_deductions    |
| +Decimal net_pay             |
| +JSON profile_snapshot       |  ← frozen at computation time
└──────────────────────────────┘
```

ADD to PAYROLL_PERIOD — status lifecycle:

→ +String status (draft / computed / finalized / released / void)

2.4.6 — Supplier Management System

✓ SUPPLIER class is correct: supplier_id, name, contact_person, phone, email, address — all match the actual database table.

✗ If this diagram shows BALE_PURCHASE_ORDER and BALE_PURCHASE_RECORD as two separate classes linked to SUPPLIER — apply the same correction as 2.4.4: merge them into one BALE_PURCHASE class.

✗ If ORDER_LINE_ITEM appears in this diagram — remove it entirely.

CORRECTED RELATIONSHIP:

```
SUPPLIER ──[1]────────[0..*]── BALE_PURCHASE
(one supplier can have many bale purchases)
```

2.4.7 — Category and Type Management System

✓ PRODUCT_CATEGORY class is correct: category_id, category_name, description — matches the categories table.

✓ The 1:many relationship from CATEGORY to PRODUCT_TYPE is correct — one category can have many types.

✗ PRODUCT_TYPE must be renamed to CATEGORY_TYPE. The actual database table is named category_types, not product_type.

✗ The relationship line from PRODUCT to CATEGORY_TYPE is shown as a solid FK line. There is NO database foreign key from products to category_types. The type is stored as plain text in the products.subcategory column and is validated by the API only.

CORRECTIONS:

→ Rename PRODUCT_TYPE → CATEGORY_TYPE

→ Add +int is_active to CATEGORY_TYPE

→ Change the PRODUCT → CATEGORY_TYPE line from SOLID to DASHED

→ Add label on dashed line: 'validated at API layer (no database FK)'

CORRECTED RELATIONSHIPS:

```
CATEGORY  ──[1]─────[0..*]──  CATEGORY_TYPE     (solid line — real FK)
PRODUCT   ──[*]─────[1]──────  CATEGORY          (solid line — real
FK)
PRODUCT   - - - - - - - - -▶  CATEGORY_TYPE     (dashed — API
validation only)
```

2.4.8 — Customer Management System

✓ CUSTOMER class with basic fields — correct.

✓ SALES_TRANSACTION (SALE) linked to CUSTOMER — correct in concept.

✓ Relationship multiplicity (one customer to many sales) — correct.

✗ CUSTOMER class is missing the Philippine address fields added by the database migration.

ADD to CUSTOMER class:

→ +String region_name

→ +String province_name

→ +String city_name

→ +String barangay_name

→ +String address_line

→ +String postal_code

These fields store the full Philippine Standard Geographic Code (PSGC) address hierarchy. They were added to support the region-province-city-barangay address selection in the customer form.

PART 3 — DOMAIN CLASS DIAGRAM (Section 4)

The Domain Class Diagram is the consolidated system-wide model. All corrections from Parts 1 and 2 apply here. This section lists every change needed and provides the corrected class structures and relationships.

Step 1 — REMOVE these classes (they do not exist in the database)

Class to Remove	Why
BALE_PURCHASE_ORDER	Not a separate table. Merged into BALE_PURCHASE (same table: bale_purchases).
ORDER_LINE_ITEM	Does not exist at all. No table, no route, no migration. Remove completely.

Step 2 — RENAME these classes

Current Name	Correct Name	Reason
BALE_PURCHASE_RECORD	BALE_PURCHASE	Same table as BALE_PURCHASE_ORDER. Merge both into this one.
PRODUCT_TYPE	CATEGORY_TYPE	Actual table name is category_types.
SALES_TRANSACTION	SALE	Actual table name is sales.
DAMAGED_ITEM	DAMAGED_INVENTORY	Actual table name is damaged_inventory.

Step 3 — ADD these classes (missing from domain diagram)

Class to Add	Key Fields	Connects to
PRODUCT	id, sku, item_code, name, category_id, subcategory, selling_price, allocated_cost, condition_grade, bale_purchase_id, stock_quantity, status	CATEGORY, CATEGORY_TYPE (dashed), BALE_PURCHASE, SALE_ITEM
SALE	id, sale_number, status (DRAFT/COMPLETED/REFUNDED/CANCELL	USER, CUSTOMER, SALE_ITEM

Class to Add	Key Fields	Connects to
	ED), clerk_id, customer_id, subtotal, tax, discount, total, receipt_no	
SALE_ITEM	id, sale_id, product_id, qty, unit_price, line_total, product_name_snapshot, sku_snapshot, barcode_snapshot	SALE, PRODUCT
INVENTORY_TRANSACTION	id, product_id, transaction_type (IN/OUT/ADJUST/RETURN), quantity, balance_after, reference, reason	PRODUCT
PERMISSION	id, name (format: module.action), description	ROLE (via role_permissions), USER (via user_permissions)

Step 4 — FIX these relationships

Relationship	Current (Wrong)	Correct
ROLE → USER	1 ROLE assigns to 0..* USER (one-to-many)	ROLE * ——— * USER (many-to-many via user_roles junction)
ROLE → permissions	"permissions" string attribute on ROLE	ROLE * ——— * PERMISSION (via role_permissions junction)
PRODUCT → CATEGORY_TYPE	Solid line (implies database FK)	Dashed line labeled "validated at API layer (no FK)"
BALE_BREAKDOWN → PRODUCT	Not shown	BALE_BREAKDOWN [1] ──generates── [0..2] PRODUCT (premium + standard only)
BALE_PURCHASE → BALE_BREAKDOWN	Not shown or shown via removed classes	BALE_PURCHASE [1] ——— [0..1] BALE_BREAKDOWN (one-to-one)
USER → EMPLOYEE	Not shown	USER [1] ——— [1] EMPLOYEE (auto-created when employee is registered)

Step 5 — CORRECTED FIELD ADDITIONS per class

BALE_PURCHASE (merged class):

```
  +String bale_batch_no, +String po_status
(PENDING/ORDERED/RECEIVED/COMPLETED/CANCELLED)
```

+int quantity_ordered, +int quantity_received

+Date expected_delivery_date, +Date actual_delivery_date

BALE_BREAKDOWN:

ADD: +int low_grade_items, +int saleable_items, +Decimal cost_per_saleable_item

CUSTOMER:

ADD: +String region_name, +String province_name, +String city_name

ADD: +String barangay_name, +String address_line, +String postal_code

USER:

REMOVE: +int role_id (no such column)

ADD: +String full_name, +String email, +int is_active

ROLE:

REMOVE: +String permissions (not a column — separate entity)

PAYROLL_PERIOD:

ADD: +String status (draft / computed / finalized / released / void)

PART 4 — MASTER SUMMARY TABLE

Use this as a quick-reference checklist when redrawing all diagrams.

Diagram	Status	Required Action
UC 2.3.1 RBAC	Minor Fix	Add User actor + Login + Logout use cases
UC 2.3.2 Inventory	Minor Fix	Change Low Stock Alert from auto-notify to user-query use case
UC 2.3.3 POS	Minor Fix	Rename cart → draft sale; split add-item and checkout; add cancel draft
UC 2.3.4 Purchasing	Moderate Fix	Remove "multiple order lines"; correct breakdown to show auto-product creation
UC 2.3.5 Payroll	Moderate Fix	Split into 5 lifecycle stages; remove owner-specific approval; fix payslip description
UC 2.3.6 Supplier	CORRECT	No changes needed
UC 2.3.7 Category	Minor Fix	Change PRODUCT → CATEGORY_TYPE line from solid to dashed
UC 2.3.8 Customer	CORRECT	No changes needed
DC 2.4.1 RBAC	Moderate Fix	Remove permissions from ROLE; remove role_id from USER; add PERMISSION class; make ROLE-USER many-to-many
DC 2.4.2 Inventory	Minor Fix	Add bale fields to PRODUCT; rename transaction class; rename damaged class
DC 2.4.3 POS	Minor Fix	Add DRAFT to SALE status; add snapshot fields to SALE_ITEM; rename clearCart
DC 2.4.4 Purchasing	Critical Fix	Remove ORDER_LINE_ITEM; merge BALE_PURCHASE_ORDER + RECORD into one class; fix BALE_BREAKDOWN fields
DC 2.4.5 Payroll	Moderate Fix	Split PAYROLL_RECORD into INPUTS + RUN_ITEM; remove PAYSLIP; remove PAYROLL_ADJUSTMENT
DC 2.4.6 Supplier	Same as 2.4.4	Merge bale classes; remove ORDER_LINE_ITEM if present
DC 2.4.7 Category	Minor Fix	Rename PRODUCT_TYPE to CATEGORY_TYPE; change PRODUCT-CATEGORY_TYPE line to dashed
DC 2.4.8 Customer	Minor Fix	Add Philippine address fields to CUSTOMER class
Domain Diagram Sec. 4	Critical Fix	Remove 2 classes; merge bale classes; add PRODUCT/SALE/SALE_ITEM/PERMISSION; fix 6 relationships

Priority order for corrections: Start with DC 2.4.4 (Purchasing) and the Domain Class Diagram — these have the most structural errors. Then fix DC 2.4.1 (RBAC) and DC 2.4.5 (Payroll). All use case diagram fixes are minor and can be done last.